INFORME DE FIN DE UNIDAD

Datos Generales

Asignatura	Base de datos
Ciclo	3 A
Unidad	1
Resultado de aprendizaje de la unidad	Modela bases de datos de problemas reales, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Nombre del Docente	Edwin René Guamán Quinche
Fecha	11/11/2026
Enlace de Git	`https://github.com/joaquinmoscol-netizen/academia-pre-db`
Nombres y apellidos del estudiante	Joaquín Moscol

1. TÍTULO

Base de datos para una academia preuniversitaria

2. CONTEXTO DEL PROBLEMA

En Loja hay academias que preparan a los chicos que salen del colegio para el examen de ingreso a la universidad. Tienen cursos por área (Salud, Ingenierías, Sociales, Económicas) en distintas jornadas, y durante el curso los estudiantes dan simulacros del examen para ver cómo van.

El problema es que las matrículas y las notas de los simulacros se llevan en cuadernos o en Excel, entonces cuesta saber quién está en cada curso, quién no ha dado los simulacros o cómo van sus puntajes. Esto le complica el trabajo a la secretaría y a los docentes, y los estudiantes no tienen un seguimiento claro. Por eso se necesita una base de datos donde estén los docentes, cursos, estudiantes, matrículas, simulacros y resultados.

3. OBJETIVOS

3.1. Objetivo General

Diseñar una base de datos relacional en MySQL para registrar los cursos, matrículas y resultados de simulacros de una academia preuniversitaria, para facilitar el seguimiento de los estudiantes.

3.2. Objetivos Específicos

- Elaborar el modelo Entidad-Relación y el modelo relacional, definiendo entidades, atributos, relaciones y claves.
- Crear la base de datos en SQL, insertar datos de prueba y resolver consultas en álgebra relacional y SQL.

4. MODELOS CONCEPTUALES DE BASES DE DATOS

4.1. Modelo Entidad-Relación (E-R)

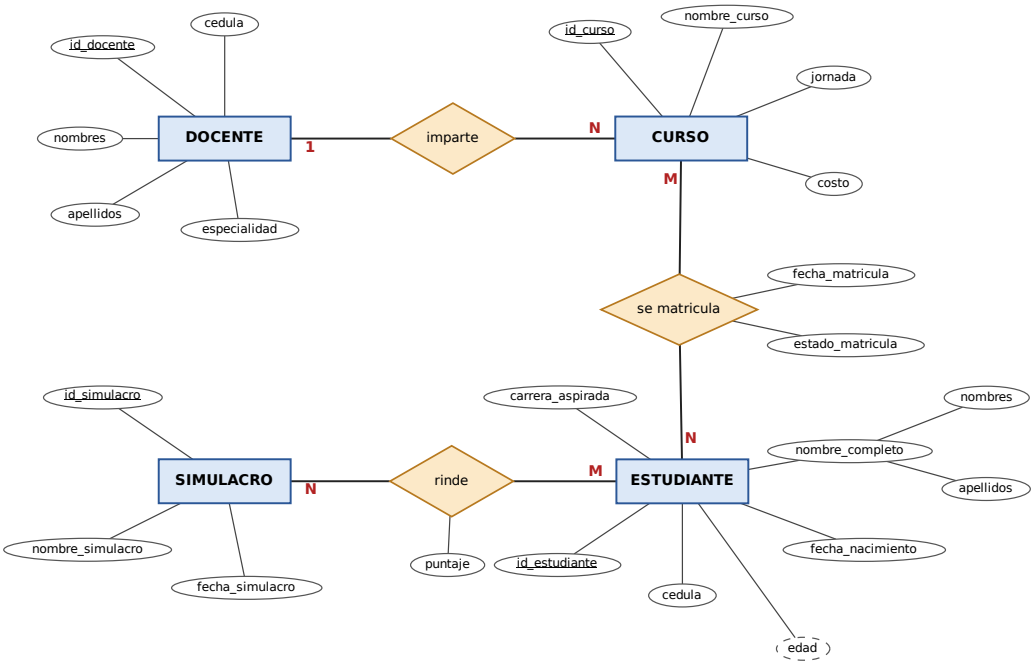


Figura 1. Diagrama Entidad-Relación de la academia preuniversitaria.

4.2. Modelo Relacional

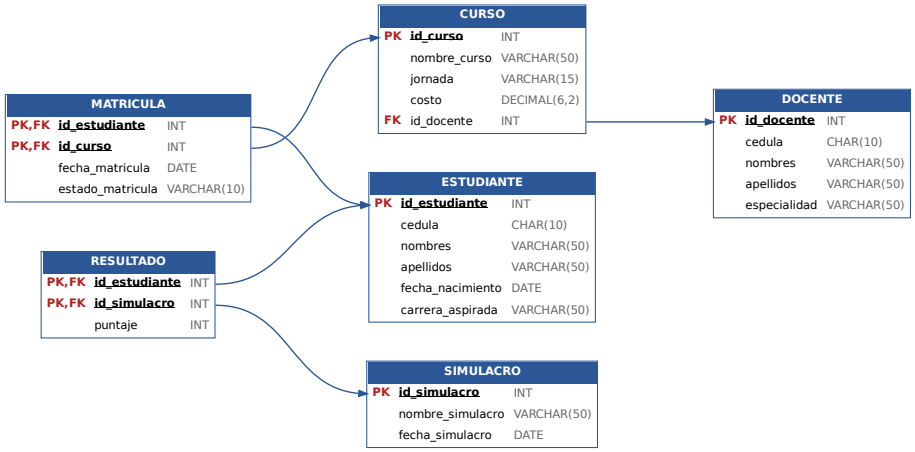


Figura 2. Modelo relacional de la academia preuniversitaria.

5. CÓDIGO SQL

5.1. Creación de Tablas (DDL)

```sql
CREATE DATABASE academia_db;
USE academia_db;

CREATE TABLE DOCENTE (
    id_docente INT NOT NULL,
    cedula VARCHAR(10) NOT NULL UNIQUE,
    nombres VARCHAR(50) NOT NULL,
    apellidos VARCHAR(50) NOT NULL,
    especialidad VARCHAR(50) NOT NULL,
    PRIMARY KEY (id_docente)
);

CREATE TABLE ESTUDIANTE (
    id_estudiante INT NOT NULL,
    cedula VARCHAR(10) NOT NULL UNIQUE,
    nombres VARCHAR(50) NOT NULL,
    apellidos VARCHAR(50) NOT NULL,
    fecha_nacimiento DATE NOT NULL,
    carrera_aspirada VARCHAR(50) NOT NULL,
    PRIMARY KEY (id_estudiante)
);

CREATE TABLE SIMULACRO (
    id_simulacro INT NOT NULL,
    nombre_simulacro VARCHAR(50) NOT NULL,
    fecha_simulacro DATE NOT NULL,
    PRIMARY KEY (id_simulacro)
);

CREATE TABLE CURSO (
    id_curso INT NOT NULL,
    nombre_curso VARCHAR(50) NOT NULL,
    jornada VARCHAR(15) NOT NULL CHECK (jornada IN ('MATUTINA', 'VESPERTINA', 'FIN_DE_SEMANA'
    )),
    costo DECIMAL(6,2) NOT NULL CHECK (costo > 0),
    id_docente INT NOT NULL,
    PRIMARY KEY (id_curso),
    FOREIGN KEY (id_docente) REFERENCES DOCENTE(id_docente)
);

CREATE TABLE MATRICULA (
    id_estudiante INT NOT NULL,
    id_curso INT NOT NULL,
    fecha_matricula DATE NOT NULL,
    estado_matricula VARCHAR(10) NOT NULL DEFAULT 'ACTIVA' CHECK (estado_matricula IN ('
    ACTIVA', 'RETIRADA')),
    PRIMARY KEY (id_estudiante, id_curso),
    FOREIGN KEY (id_estudiante) REFERENCES ESTUDIANTE(id_estudiante),
    FOREIGN KEY (id_curso) REFERENCES CURSO(id_curso)
);

CREATE TABLE RESULTADO (
```

```
    id_estudiante INT NOT NULL,
    id_simulacro INT NOT NULL,
    puntaje INT NOT NULL CHECK (puntaje BETWEEN 0 AND 1000),
    PRIMARY KEY (id_estudiante, id_simulacro),
    FOREIGN KEY (id_estudiante) REFERENCES ESTUDIANTE(id_estudiante),
    FOREIGN KEY (id_simulacro) REFERENCES SIMULACRO(id_simulacro)
);
```

5.2. Inserción de Datos (DML)

```
USE academia_db;

INSERT INTO DOCENTE VALUES
 (1, '1104567890', 'Carmen', 'Jaramillo', 'Matematica'),
 (2, '1103456789', 'Luis', 'Ordonez', 'Fisica'),
 (3, '1105678901', 'Patricia', 'Cueva', 'Biologia'),
 (4, '1102345678', 'Diego', 'Riofrio', 'Lenguaje'),
 (5, '1106789012', 'Andrea', 'Valdivieso', 'Razonamiento');

INSERT INTO ESTUDIANTE VALUES
 (1, '1150123456', 'Valentina', 'Torres', '2008-03-14', 'Medicina'),
 (2, '1150234567', 'Mateo', 'Castillo', '2007-11-02', 'Ingenieria Civil'),
 (3, '1150345678', 'Camila', 'Ojeda', '2008-06-21', 'Medicina'),
 (4, '1150456789', 'Sebastian', 'Pineda', '2008-01-30', 'Computacion'),
 (5, '1150567890', 'Daniela', 'Romero', '2007-09-09', 'Odontologia'),
 (6, '1150678901', 'Andres', 'Guerrero', '2007-12-17', 'Derecho'),
 (7, '1150789012', 'Sofia', 'Armijos', '2008-04-05', 'Enfermeria'),
 (8, '1150890123', 'Joel', 'Sarango', '2007-08-12', 'Economia');

INSERT INTO SIMULACRO VALUES
 (1, 'Diagnostico', '2026-09-19'),
 (2, 'Parcial 1', '2026-10-10'),
 (3, 'Parcial 2', '2026-10-31');

INSERT INTO CURSO VALUES
 (1, 'Ciencias de la Salud', 'MATUTINA', 450.00, 1),
 (2, 'Ciencias de la Salud', 'VESPERTINA', 450.00, 3),
 (3, 'Ingenierias', 'MATUTINA', 420.00, 2),
 (4, 'Ciencias Sociales', 'FIN_DE_SEMANA', 350.00, 4),
 (5, 'Ciencias Economicas', 'VESPERTINA', 350.00, 5);

INSERT INTO MATRICULA VALUES
 (1, 1, '2026-08-20', 'ACTIVA'),
 (2, 3, '2026-08-21', 'ACTIVA'),
 (3, 1, '2026-08-22', 'ACTIVA'),
 (4, 3, '2026-08-25', 'ACTIVA'),
 (5, 2, '2026-08-26', 'ACTIVA'),
 (6, 4, '2026-08-28', 'ACTIVA'),
 (7, 2, '2026-09-01', 'RETIRADA'),
 (8, 5, '2026-09-02', 'ACTIVA');

INSERT INTO RESULTADO VALUES
 (1, 1, 712), (1, 2, 768), (1, 3, 805),
 (2, 1, 690), (2, 2, 735), (2, 3, 781),
 (3, 1, 640), (3, 2, 702), (3, 3, 755),
 (4, 1, 598), (4, 2, 655),
```

```
  (5, 1, 720), (5, 2, 742), (5, 3, 790),
  (7, 1, 540);
```

6. CONSULTAS DE ÁLGEBRA RELACIONAL Y TRADUCCIÓN A SQL

Consulta 1

Estudiantes activos del curso de Ciencias de la Salud en la jornada matutina.

Expresión en álgebra relacional:

```
π nombres, apellidos, carrera_aspirada (σ nombre_curso == 'Ciencias de la Salud' and jornada ==
'MATUTINA' and estado_matricula == 'ACTIVA' (estudiante ⋈ matricula ⋈ curso))
```

SQL:

```sql
SELECT nombres, apellidos, carrera_aspirada
FROM ESTUDIANTE
NATURAL JOIN MATRICULA
NATURAL JOIN CURSO
WHERE nombre_curso = 'Ciencias de la Salud'
  AND jornada = 'MATUTINA'
  AND estado_matricula = 'ACTIVA';
```

Captura de la consulta 1 en Relational

Consulta 2

Estudiantes activos que todavía no rinden ningún simulacro.

Expresión en álgebra relacional:

```
π nombres, apellidos ((π id_estudiante (σ estado_matricula == 'ACTIVA' (matricula)) - π
id_estudiante (resultado)) ⋈ estudiante)
```

SQL:

```sql
SELECT nombres, apellidos
FROM ESTUDIANTE
NATURAL JOIN MATRICULA
WHERE estado_matricula = 'ACTIVA'
  AND id_estudiante NOT IN (SELECT id_estudiante FROM RESULTADO);
```

Captura de la consulta 2 en Relational

Consulta 3

Estudiantes que rindieron todos los simulacros.

Expresión en álgebra relacional:

```
π nombres, apellidos ((π id_estudiante, id_simulacro (resultado) ÷ π id_simulacro (simulacro)) ⋈
estudiante)
```

SQL:

```sql
SELECT nombres, apellidos
FROM ESTUDIANTE e
WHERE NOT EXISTS (
    SELECT * FROM SIMULACRO s
    WHERE NOT EXISTS (
        SELECT * FROM RESULTADO r
        WHERE r.id_estudiante = e.id_estudiante
          AND r.id_simulacro = s.id_simulacro));
```

Captura de la consulta 3 en Relational

Consulta 4

Estudiantes activos de la jornada matutina que sacaron 750 o más en algún simulacro.

Expresión en álgebra relacional:

```
π nombres, apellidos ((π id_estudiante (σ puntaje >= 750 (resultado)) ∩ π id_estudiante (σ jornada
== 'MATUTINA' and estado_matricula == 'ACTIVA' (matricula ⋈ curso))) ⋈ estudiante)
```

SQL:

```sql
SELECT nombres, apellidos
FROM ESTUDIANTE
WHERE id_estudiante IN (SELECT id_estudiante FROM RESULTADO WHERE puntaje >= 750)
  AND id_estudiante IN (SELECT id_estudiante FROM MATRICULA NATURAL JOIN CURSO
                        WHERE jornada = 'MATUTINA' AND estado_matricula = 'ACTIVA');
```

Captura de la consulta 4 en Relational

Consulta 5

Docente y estudiantes activos del curso de Ciencias de la Salud matutino en una sola lista.

Expresión en álgebra relacional:

```
π nombres, apellidos (σ nombre_curso == 'Ciencias de la Salud' and jornada == 'MATUTINA' (docente ⋈
curso)) ∪ π nombres, apellidos (σ nombre_curso == 'Ciencias de la Salud' and jornada == 'MATUTINA'
and estado_matricula == 'ACTIVA' (estudiante ⋈ matricula ⋈ curso))
```

SQL:

```sql
SELECT nombres, apellidos
FROM DOCENTE
NATURAL JOIN CURSO
WHERE nombre_curso = 'Ciencias de la Salud' AND jornada = 'MATUTINA'
UNION
SELECT nombres, apellidos
FROM ESTUDIANTE
NATURAL JOIN MATRICULA
NATURAL JOIN CURSO
WHERE nombre_curso = 'Ciencias de la Salud' AND jornada = 'MATUTINA'
  AND estado_matricula = 'ACTIVA';
```

Captura de la consulta 5 en Relational

CONCLUSIONES

Hacer el diagrama Entidad-Relación me ayudó a ordenar cómo funciona la academia antes de pensar en tablas. Lo más difícil fue definir las cardinalidades y darme cuenta de que la matrícula y los resultados de los simulacros son relaciones de muchos a muchos, por lo que cada una termina siendo una tabla con dos claves foráneas.

Al escribir el código SQL vi para qué sirven las restricciones. Con las claves foráneas no se puede registrar una matrícula de un estudiante que no existe, y con CHECK no se puede guardar un puntaje mayor a 1000 o una jornada que no sea válida.

El álgebra relacional sirvió para pensar primero qué datos necesitaba y después escribir el SQL. Las consultas que más me costaron fueron la división y la diferencia, pero también fueron las más útiles, por ejemplo para saber qué estudiantes dieron todos los simulacros y cuáles no han dado ninguno.

BIBLIOGRAFÍA

[1] R. Elmasri y S. B. Navathe, *Fundamentos de Sistemas de Bases de Datos*, 7.ª ed. Madrid, España: Pearson, 2016.

[2] Oracle, “MySQL 8.0 Reference Manual.” [En línea]. Disponible: `https://dev.mysql.com/doc/refman/8.0/en/`. (Accedido: 11 de noviembre de 2026).

[3] S. Tomasi, “Relational.” [En línea]. Disponible: `https://ltworf.github.io/relational/`. (Accedido: 11 de noviembre de 2026).